Task 1:

See graph.py and graphtester.py

Task 2:

BFS works by taking some given node to start with (we will say it is at level 0), and assuming that any neighbours of that node are found and are at the next level (in this case, 1), then any neighbours of those neighbours which were not already found are at the next level (2) and so on. This means that every visitable node will be visited, as we account for every neighbour of a node, every neighbour of those neighbours etc. until every neighbour has been visited, leaving us with a full list of nodes visitable from the node we started at. This method of traversal is logically based upon a tree, where we look at every branch of a node and every branch of those branches and no node is repeated. Similarly, though we have an edge between D and E, we do not traverse it as D is already “visited” from node C, which would be equivalent to D being a branch of C but not E. This makes for more efficient traversal of the graph, having not needed to traverse every edge in it.

Task 3:

The result of BFS on the graph is [A,B,C,E,F,D], whereas DFS gives [A,B,C,D,E,F]. This is because, while BFS searches by level, DFS searches by first ‘branch’ if we continue the idea of the tree representation of the graph. Instead of visiting B and then each branch of B, we visit B, then the first branch of B, then the first of that branch and so on. Because this graph has a cycle starting at B, this means we go from B to E, then once we realise that B has been visited, we look at the next branch of B (this is E, so we do not visit) before the final branch in F.

Task 4:

See graph.py and graphtester.py

Task 5:

Weights in graphs are important for determining the order for edge traversal. In previous examples of BFS and DFS, edge traversal was determined by whichever vertex was added first to the current node’s list of neighbours. With weights however, which edge is traversed can be determined based on the value of the weight (for instance, we may want to traverse the edge with the smallest weight). This can have benefits when applied to real world scenarios. For example, the given graph is directed, and we can traverse from A to B, B to C and A to C. If these three nodes were locations on a map, and the weight was the travelling time between them, we now know the time to travel between A and C via B or between A and C. This is just one example, but it demonstrates the benefit that adding weights provides by giving an order to which edges are traversed other than the order in which they were added.

Task 6:

See graphvisualiser.py

Task 7:

Results

```
/Users/alexr/PycharmProjects/Software
Added 10 vertices in 0.0000 seconds
Added 26 edges in 0.0048 seconds
BFS completed in 0.0000 seconds

Added 100 vertices in 0.0000 seconds
Added 296 edges in 0.0002 seconds
BFS completed in 0.0003 seconds

Added 1000 vertices in 0.0002 seconds
Added 2998 edges in 0.0027 seconds
BFS completed in 0.0222 seconds
```

```
/Users/alexr/PycharmProjects/Software
Added 10 vertices in 0.0000 seconds
Added 28 edges in 0.0023 seconds
BFS completed in 0.0000 seconds

Added 100 vertices in 0.0000 seconds
Added 298 edges in 0.0003 seconds
BFS completed in 0.0003 seconds

Added 1000 vertices in 0.0002 seconds
Added 2997 edges in 0.0028 seconds
BFS completed in 0.0231 seconds
```

```
/Users/alexr/PycharmProjects/Software
Added 10 vertices in 0.0000 seconds
Added 27 edges in 0.0031 seconds
BFS completed in 0.0000 seconds

Added 100 vertices in 0.0000 seconds
Added 297 edges in 0.0002 seconds
BFS completed in 0.0003 seconds

Added 1000 vertices in 0.0002 seconds
Added 2999 edges in 0.0029 seconds
BFS completed in 0.0223 seconds
```

As can be seen from these screenshots, in terms of adding vertices, there is no variation in times between runs of the program, due to the low complexity of O(N). There appears to be much higher time for the 30-edge graph than the 300-edge graph, however this is due to the random number operations needing time to start (as we can see in the screenshot below, the time drops back to a more logical number with a second iteration). The time increases substantially between adding vertices and adding edges for a 3000-edge graph, as the add_edge function has more operations and assignments and therefore a higher complexity. BFS is completed almost instantly for the two smaller sizes, but takes a consistent amount of time for the 3000-edge graph. This is due to the nature of the graph and BFS. With a randomly generated graph of large size, the likelihood of the node being searched for sitting at a very low

level is decreased, and the search is likely to have to sort through a large number of nodes at the same level, resulting in a consistent time for the operation to occur. BFS could be slowed by a large number of different levels for nodes, but this is unlikely to occur for a randomly generated graph. As a result, BFS at larger numbers has a very consistent execution time.

```
Added 10 vertices in 0.0000 seconds
Added 29 edges in 0.0023 seconds
BFS completed in 0.0000 seconds

Added 100 vertices in 0.0000 seconds
Added 299 edges in 0.0002 seconds
BFS completed in 0.0003 seconds

Added 1000 vertices in 0.0002 seconds
Added 2998 edges in 0.0027 seconds
BFS completed in 0.0228 seconds

Added 10 vertices in 0.0000 seconds
Added 29 edges in 0.0000 seconds
BFS completed in 0.0000 seconds

Added 100 vertices in 0.0000 seconds
Added 295 edges in 0.0002 seconds
BFS completed in 0.0002 seconds

Added 1000 vertices in 0.0001 seconds
Added 2999 edges in 0.0022 seconds
BFS completed in 0.0191 seconds
```

Task 8:

See graphchallenge.py